

Moving the VPS file storage from FTP to S3

Time Tracker project · plan for the owner and whoever manages the VPS · prepared 26 September 2026

1. Summary and decisions

The office has a separate VPS that stores files and images, reached today over FTP. FTP logs in again for almost every file it fetches, so it is slow for many small files. Phase 10 of the Time Tracker adds screenshots (tens of thousands of small images), and the Spatie Media Library that stores them can use any Laravel filesystem disk. This plan makes the VPS speak the **S3 protocol** by installing **MinIO**, an S3-compatible server, behind HTTPS.

Decision	Choice
S3 service	MinIO, self-hosted on the existing VPS
VPS operating system	Linux (Ubuntu or Debian); commands below use <code>apt</code> , <code>systemd</code> and <code>apache2</code>
Existing files on FTP	Move them to S3 too (section 7)
Laravel side	One disk called <code>screenshots</code> , chosen by <code>SCREENSHOT_DISK: local</code> while developing, <code>s3</code> in production. Originals and thumbnails both go to that disk.
Screenshot library	Spatie Media Library v11 (unchanged: it only sees a Laravel disk)
Estimated effort	About half a day for a clean install and tests; migration time for existing files depends on their size

The target picture

```
Laravel app (production) --HTTPS--> Apache on the VPS --> MinIO (127.0.0.1:9000) -->
dedicated data disk
Desktop agents           --HTTPS--> Laravel API   (agents never talk to the VPS directly)
Dashboard images         --HTTPS--> Laravel API   (permission check, then streamed from S3)
```

What does not change: the desktop app, the dashboard and the API routes. Only the disk driver behind `screenshots` changes, and it changes by editing production `.env` values.

Read this first, MinIO distribution: in 2025 MinIO changed how its community edition is distributed (source releases, a reduced web console, container images no longer regularly published). Before installing, confirm the current official download for the Linux server binary. If it is not available or not trusted, use **Garage** instead. Sections 4 to 9 stay the same because both speak S3.

Contents

1. Summary and decisions
2. Before you start
3. Install MinIO on the VPS
4. HTTPS with Apache and Let's Encrypt, and the firewall
5. Buckets, users and permissions
6. Connect Laravel (matches Phase 10)
7. Move the existing files from FTP to S3
8. Backups and monitoring
9. Verification checklist
10. Rollback plan and risks
11. Appendix: files and commands in one place, timeline

2. Before you start

Collect these facts first. Nothing here changes the VPS.

- ☐ VPS size: CPU, RAM, and **free disk**. Screenshots need about 2 GB per person per year at a 10-minute interval (about 40 to 80 GB a year for 20 people), on top of the existing files.
- ☐ Which disk or volume will hold the data. Use a **dedicated** disk or volume (XFS or ext4), not the system disk.
- ☐ The S3 web address, for example `s3.<office-domain>`, and access to its DNS.
- ☐ An admin SSH account with `sudo`, using a key.
- ☐ Everything that reads or writes the FTP storage today: every application, script and person. Write down each one and how it connects.
- ☐ Total size and number of files on FTP, per top-level folder (for example `find /path -type f | wc -l` and `du -sh /path/*`).
- ☐ Which web server owns ports 80 and 443 on the VPS (Apache, as understood so far; confirm with `sudo ss -tlnp | grep -E ':80|:443'`), and which sites it already serves.
- ☐ A password manager entry for the MinIO root credentials and one for each app key.

Names used in this plan (replace them)

Placeholder	Meaning
<code><s3-domain></code>	The public S3 name, e.g. <code>s3.example.org</code>
<code><data-dir></code>	Folder on the dedicated disk, e.g. <code>/srv/minio/data</code>
<code><root-user></code> / <code><root-password></code>	MinIO admin login: kept private, never used by an application
<code><app-key></code> / <code><app-secret></code>	The key pair created for one application

3. Install MinIO on the VPS

3.1 A service account and the data folder

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin minio-user
sudo mkdir -p <data-dir>
sudo chown -R minio-user:minio-user <data-dir>
sudo chmod 750 <data-dir>
```

3.2 The server and the client

Download the Linux server binary (`minio`) and the client (`mc`) from MinIO's official download site, as noted in the warning on page 1. Verify the checksum published next to each download, then:

```
sudo install -m 755 minio /usr/local/bin/minio
sudo install -m 755 mc /usr/local/bin/mc
minio --version
```

3.3 The settings file

Create `/etc/default/minio` (owned by root, mode 600) with a long random root password (for example `openssl rand -base64 32`):

```
MINIO_ROOT_USER=<root-user>
MINIO_ROOT_PASSWORD=<root-password>
MINIO_VOLUMES="<data-dir>"
MINIO_OPTS="--address 127.0.0.1:9000 --console-address 127.0.0.1:9001"
MINIO_SERVER_URL="https://<s3-domain>"
```

Both ports listen on **127.0.0.1 only**: nothing can reach MinIO except through Apache on the same machine.

3.4 A systemd service that starts on boot

```
[Unit]
Description=MinIO
After=network-online.target
Wants=network-online.target

[Service]
User=minio-user
Group=minio-user
EnvironmentFile=/etc/default/minio
ExecStart=/usr/local/bin/minio server $MINIO_OPTS $MINIO_VOLUMES
Restart=always
LimitNOFILE=1048576
TimeoutStopSec=infinity

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now minio
systemctl status minio
curl -I http://127.0.0.1:9000/minio/health/live      # expect HTTP 200
```

4. HTTPS with Apache and Let's Encrypt, and the firewall

The VPS already runs **Apache**, so Apache is used as the front door for MinIO. Do not add nginx next to it: two web servers cannot share ports 80 and 443. The S3 address becomes one more virtual host beside the sites Apache already serves, and nothing existing changes.

Check first: `sudo ss -tlnp | grep -E ' :80|:443 '` shows which program owns those ports. If it is Apache (`apache2` on Ubuntu/Debian), follow this section. If it is nginx, the same settings apply with nginx syntax (see the note at the end of this section).

4.1 DNS, modules and certificate

Create an `A` record for `<s3-domain>` pointing at the VPS. Then:

```
sudo a2enmod proxy proxy_http ssl headers rewrite
sudo apt install certbot python3-certbot-apache
sudo systemctl reload apache2
```

Create a plain HTTP virtual host for the name first (certbot needs it to prove ownership), then get the certificate:

```
# /etc/apache2/sites-available/<s3-domain>.conf
<VirtualHost *:80>
    ServerName <s3-domain>
    DocumentRoot /var/www/html
</VirtualHost>
```

```
sudo a2ensite <s3-domain>
sudo systemctl reload apache2
sudo certbot --apache -d <s3-domain>
```

Certbot adds the HTTPS host and a redirect from HTTP, and installs a timer that renews the certificate automatically. Check it once with `sudo certbot renew --dry-run`.

4.2 The Apache virtual host

S3 clients sign each request including the `Host` header and the exact file path, so Apache must pass both through unchanged and must not limit or rewrite large uploads. Edit the HTTPS file that certbot created (`<s3-domain>-le-ssl.conf`) so that it contains:

```
<VirtualHost *:443>
    ServerName <s3-domain>

    # SSLCertificateFile and SSLCertificateKeyFile lines were added by certbot
    Header always set Strict-Transport-Security "max-age=31536000"

    ProxyRequests Off

    # the Host header reaches MinIO unchanged (the request signature depends on it)
    ProxyPreserveHost On

    # file names with encoded slashes are not altered
    AllowEncodedSlashes NoDecode

    # no upload size limit here: MinIO enforces its own
    LimitRequestBody 0
    ProxyTimeout 300

    ProxyPass          / http://127.0.0.1:9000/ nocanon
    ProxyPassReverse / http://127.0.0.1:9000/
    RequestHeader set X-Forwarded-Proto "https"

    ErrorLog ${APACHE_LOG_DIR}/<s3-domain>-error.log
    CustomLog ${APACHE_LOG_DIR}/<s3-domain>-access.log combined
</VirtualHost>
```

```
sudo apachectl configtest && sudo systemctl reload apache2
```

Apache accepts comments only on their own line, never after a directive on the same line, so keep them as shown.

`nocanon` and `AllowEncodedSlashes NoDecode` are the two settings people most often forget: without them, some file names fail with a "signature does not match" (403) error even though the keys are right.

If the VPS uses nginx instead: the equivalent settings are `proxy_set_header Host $http_host`, `proxy_buffering off`, `proxy_request_buffering off`, `client_max_body_size 0` and `proxy_pass http://127.0.0.1:9000` with no URI part, plus certbot's nginx plugin.

4.3 The web console is not published

Port 9001 (the admin web page) is not exposed. Reach it from your own computer through an SSH tunnel when needed:

```
ssh -L 9001:127.0.0.1:9001 <admin-user>@<vps>          # then open http://127.0.0.1:9001
```

4.4 Firewall

```
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw allow 80/tcp      # only for certificate renewal and the redirect
sudo ufw enable
sudo apt install fail2ban  # protects SSH from repeated guesses
```

Ports 9000 and 9001 stay closed to the outside. If the VPS provider has its own firewall panel, apply the same rules there.

5. Buckets, users and permissions

A **bucket** is a top-level container, like a folder with its own permissions. Use one bucket per application, so a leaked key can only touch one bucket.

5.1 Connect the client to your server

```
mc alias set office https://<s3-domain> <root-user> <root-password>
mc admin info office
```

5.2 Create the buckets

```
mc mb office/screenshots          # for the Time Tracker
mc mb office/<other-bucket>        # one for each collection of existing files found in section 2
mc anonymous set none office/screenshots  # no public access, ever
```

5.3 One policy and one key pair per application

Save this as `screenshots-policy.json` (it allows only this bucket):

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": ["arn:aws:s3:::screenshots"] },
    { "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": ["arn:aws:s3:::screenshots/*"] }
  ]
}
```

```
mc admin policy create office screenshots-rw screenshots-policy.json
mc admin user add office <app-key> <app-secret>
mc admin policy attach office screenshots-rw --user <app-key>
```

Store `<app-key>` and `<app-secret>` only in the application's production `.env` and in the password manager. Repeat section 5.3 for every other bucket with its own policy and its own key.

5.4 Optional protection

- **Versioning** (`mc version enable office/screenshots`) keeps an older copy if a file is overwritten or deleted by mistake. It uses more disk; decide with the disk budget in mind.
- The Time Tracker itself never deletes screenshots (they are kept permanently), so the app key could be limited to get, put and list. Keep `DeleteObject` only if the app must be able to clean up a failed upload.

6. Connect Laravel (matches Phase 10)

This is what the developer changes in `apps/api` when Phase 10 is built. It is listed here so the VPS and the code stay in step.

6.1 Package

```
composer require league/flysystem-aws-s3-v3 "^3.0"
```

6.2 The `screenshots` disk in `config/filesystems.php`

```
'screenshots' => [
    'driver' => env('SCREENSHOT_DISK', 'local') === 's3' ? 's3' : 'local',
    // local (development and browser tests)
    'root' => storage_path('app/private/screenshots'),
    // s3 (production)
    'key' => env('AWS_ACCESS_KEY_ID'),
    'secret' => env('AWS_SECRET_ACCESS_KEY'),
    'region' => env('AWS_DEFAULT_REGION', 'us-east-1'),
    'bucket' => env('AWS_BUCKET'),
    'endpoint' => env('AWS_ENDPOINT'),
    'use_path_style_endpoint' => true,
    'visibility' => 'private',
    'throw' => true,
],
```

Simpler alternative: define two disks, `screenshots_local` and `screenshots_s3`, and let `SCREENSHOT_DISK` name the one to use. Either way, the code only ever refers to the disk name from the environment.

6.3 Production `.env` (never committed)

```
SCREENSHOT_DISK=s3
AWS_ACCESS_KEY_ID=<app-key>
AWS_SECRET_ACCESS_KEY=<app-secret>
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=screenshots
AWS_ENDPOINT=https://<s3-domain>
AWS_USE_PATH_STYLE_ENDPOINT=true
```

`use_path_style_endpoint` matters: without it the client builds addresses like `screenshots.<s3-domain>`, which need extra DNS and a wildcard certificate.

6.4 Spatie Media Library

In `config/media-library.php` set `disk_name` and `conversions_disk` to `screenshots`. Thumbnails are built by a conversion job; in production that job runs on the queue, so `queue:work` must run under a supervisor (or systemd). In development and the browser tests the queue is `sync`.

6.5 Apply and smoke-test

```
php artisan config:cache
php artisan tinker
>>> Storage::disk('screenshots')->put('smoke/test.txt', 'ok');
>>> Storage::disk('screenshots')->get('smoke/test.txt');          // "ok"
>>> Storage::disk('screenshots')->delete('smoke/test.txt');
```

Files are never public: the dashboard receives them through the API after the hierarchy check, not through a bucket URL.

7. Move the existing files from FTP to S3

The safest way is to copy first, verify, switch each application, and keep FTP read-only as the way back.

7.1 Order of work

1. **Inventory** (section 2): size and file count per top-level folder, and who uses each one.
2. **One bucket per collection** (section 5). Keep the **same folder paths as object keys**, so any path stored in a database still works after the move.
3. **Install rclone** on the VPS and set up two remotes, `ftp` and `office` (the S3 side):

```
sudo apt install rclone
rclone config      # new remote "ftp": type ftp, host, user, password, explicit TLS if the
FTP server offers it
                  # new remote "office": type s3, provider Minio, endpoint https://<s3-
domain>, key and secret of an admin key
```

4. **Dry run**, then the real copy (rclone copies, it does not delete the source):

```
rclone copy ftp:/path/folder office:<bucket> --dry-run
rclone copy ftp:/path/folder office:<bucket> --transfers 8 --checkers 16 --progress --log-
file=copy.log --log-level INFO
```

5. **Verify**. FTP servers usually give no checksums, so compare sizes and spot-check content:

```
rclone check ftp:/path/folder office:<bucket> --size-only --one-way
rclone md5sum office:<bucket> | shuf -n 20      # then download 20 of them from FTP and
compare by hand
```

6. **Catch-up copy** just before the switch (repeat step 4; only new or changed files are sent).
7. **Switch one application at a time**, during a quiet period: stop writes to that folder, run the catch-up copy, change that application's storage setting to S3 (Laravel apps: the disk configuration and `.env`), test reading and writing, resume.
8. **Keep FTP read-only for 30 days** as the rollback, then archive it and remove access.

7.2 Applications that are not Laravel

Before moving a folder, confirm that every application using it can talk S3 (most languages have an S3 library). If one cannot, leave that folder on FTP for now, or put a small adapter in front of it, and record it as an open item in section 10.

7.3 Checklist for each folder

- ☐ Bucket created, key and policy limited to it
- ☐ Dry run clean, copy finished, size check passed, 20-file spot check passed
- ☐ Catch-up copy done, application switched, read and write tested
- ☐ FTP set read-only, removal date written down (30 days)

8. Backups and monitoring

8.1 Backups (S3 storage is not a backup by itself)

- A nightly copy of every bucket to a **different machine or location** (not the same VPS disk):

```
mc mirror --overwrite office/screenshots /backup/screenshots      # or: rclone sync
office:screenshots backupremote:screenshots
```

Run it from cron or a systemd timer. Use `--remove` only if the backup should mirror deletions; for screenshots, which are never deleted, leave it out.

- Or use the VPS provider's disk snapshots, as long as they are stored away from the VPS.
- Do one **restore test** (copy a bucket back into a scratch bucket and open some files) and write the steps down.
- The MySQL backup still matters: it holds the record of each screenshot (who, when), while S3 holds the image.

8.2 Monitoring

What	How
Service is up	<code>https://<s3-domain>/minio/health/live</code> from your uptime monitor
Disk space	Alert at 80% on the data disk; extend the disk or add storage before 90%
Certificate	Alert when it has fewer than 14 days left (the certbot timer should renew at 30)
Bucket size growth	<code>mc du office/screenshots</code> monthly, compared with the estimate in section 2
Logs	<code>journalctl -u minio</code> ; Apache logs for the S3 host in <code>/var/log/apache2/</code> , rotated by <code>logrotate</code>
Updates	Monthly check for MinIO and Ubuntu/Debian security updates; restart in a quiet period
Key rotation	Once a year, or when someone with access leaves: create a new key pair, update the app's <code>.env</code> , run <code>php artisan config:cache</code> , then remove the old key

9. Verification checklist

Do these in order. Write the result and the date next to each line.

- ☐ MinIO starts by itself after a VPS restart (`sudo reboot` , then `systemctl status minio`)
- ☐ Upload and download a test file with `mc cp` through `https://<s3-domain>`
- ☐ Upload, read and delete a test file from Laravel with the tinker commands in section 6.5
- ☐ An anonymous request to a file in a bucket is refused (403) — `curl -I https://<s3-domain>/screenshots/anything`
- ☐ The app key for one bucket cannot list or read another bucket
- ☐ Ports 9000 and 9001 are not reachable from the internet (test from outside: `nc -vz <vps-ip> 9000` fails)
- ☐ The web console works only through the SSH tunnel; Apache does not publish port 9001
- ☐ Certificate is valid and `certbot renew --dry-run` passes
- ☐ **Speed test**: fetch the same 100 small files from FTP and from S3 and record both times (expect S3 to be several times faster)
- ☐ Backup runs, and one restore test is done and documented
- ☐ After Phase 10 is built: one real screenshot goes all the way (desktop app, then API, then S3, then the dashboard gallery), and the thumbnail is created
- ☐ The browser test suite still passes with `SCREENSHOT_DISK=local` (development stays unchanged)

10. Rollback plan and risks

Rollback

- **Screenshots (new data):** set `SCREENSHOT_DISK` back and run `php artisan config:cache`. Anything already stored in S3 stays there; nothing is lost. (Going back to FTP for screenshots would need the FTP disk added again, so treat S3 as one-way once screenshots are live.)
- **Existing files:** FTP stays read-only for 30 days and is untouched by rclone, so switching an application's disk back to FTP restores the old state. New files written to S3 since the switch would have to be copied back (`rclone copy office:<bucket> ftp:/path`).
- Do not delete anything on FTP until the 30 days have passed and every application has been running on S3 without problems.

Risks

Risk	What could happen	What we do
MinIO distribution change	Official binary not available, or only source builds	Confirm the download first (page 1); Garage as the fallback; both speak S3
Single VPS	If the VPS or its disk fails, files are lost	Off-box nightly backup (section 8); restore tested once
Disk full	Uploads fail; desktop apps keep shots locally and retry	80% alert; capacity review each quarter
An app cannot speak S3	That folder cannot be moved	Leave on FTP for now and record it; add a library or adapter later
Wrong <code>Host</code> header or path style	Signature errors (403) from Laravel	Use the Apache virtual host in section 4.2 and <code>use_path_style_endpoint=true</code>
Certificate expires	All S3 calls fail	Certbot timer plus the 14-day alert
Leaked key	One bucket exposed	Per-app key and policy; rotate the key; never use the root key in an app
Extra service to maintain	Unpatched software	Monthly update check (section 8.2)

11. Appendix

A. Files and where they go

File	Purpose
<code>/etc/default/minio</code> (root, mode 600)	MinIO settings and root login (section 3.3)
<code>/etc/systemd/system/minio.service</code>	Service definition (section 3.4)
<code>/etc/apache2/sites-available/<s3-domain>.conf</code> and <code><s3-domain>-le-ssl.conf</code>	HTTPS proxy (section 4.2)
<code>screenshots-policy.json</code>	Bucket policy (section 5.3)
<code>apps/api/config/filesystems.php</code>	The <code>screenshots</code> disk (section 6.2)
production <code>.env</code> on the app server	Keys and the endpoint (section 6.3); never committed

B. Everyday commands

```
mc ls office                # list buckets
mc ls office/screenshots --recursive | head
mc du office/screenshots    # size of a bucket
mc admin info office        # server health and disk usage
mc admin user list office   # who has keys
sudo systemctl restart minio
sudo journalctl -u minio -f # live log
```

C. Timeline

When	Work
Day 0	Collect the facts (section 2); confirm the MinIO download; DNS record
Day 1	Install MinIO, Apache virtual host, certificate, firewall; create buckets, policies and keys (sections 3 to 5)
Day 2	Connect Laravel with the local-to-S3 switch and run the smoke test (section 6); verification checklist (section 9)
Day 3 onwards	Copy and switch existing files one application at a time (section 7)
Backups	Set up before any real data is stored, and test one restore
Day 30 after the last switch	Retire FTP

All keys and passwords in this document are placeholders in angle brackets. Real values belong only in the password manager and in each application's production `.env`.